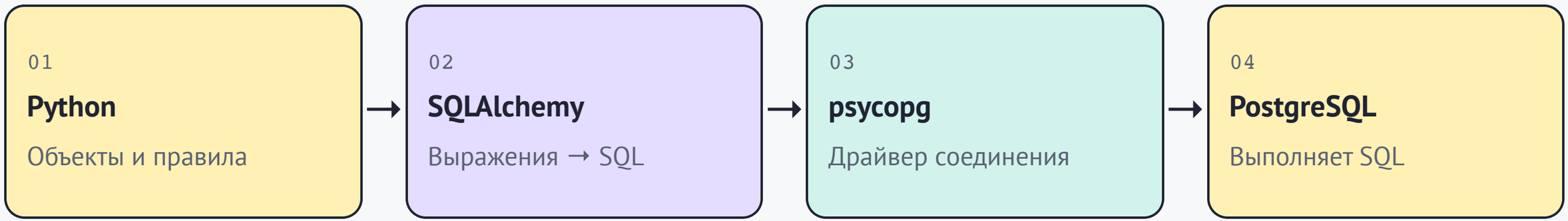
Как Python разговаривает с базой

SQLAlchemy связывает объекты Python и SQL-запросы.



`postgresql+psycopg://…`	База PostgreSQL + явно выбранный драйвер psycopg

↳ ORM упрощает работу с таблицами через объекты. SQL и ограничения базы продолжают действовать.

ORM-модель описывает таблицу

Класс Product связывает атрибуты Python со столбцами products.

models.py · сокращённый фрагмент — Учебный пример

```
class Product(Base):
    __tablename__ = "products"
    id: Mapped[int] = mapped_column(
        Identity(), primary_key=True)
    name: Mapped[str] =
mapped_column(Text)
    price: Mapped[Decimal] =
mapped_column(
        Numeric(10, 2))
```

Python	PostgreSQL
Product	products
id: int	integer · PRIMARY KEY
name: str	text · NOT NULL
price: Decimal	numeric(10,2)

↳ Класс задаёт соответствие. Таблицу в нашем приложении создаёт миграция.

ForeignKey и relationship: две роли

Ссылка защищает данные. Связь объектов помогает читать Python-код.

Product · фрагмент модели — Учебный пример

```
brand_id: Mapped[int] = mapped_column(
    ForeignKey("brands.id"))

brand: Mapped[Brand] = relationship(
    back_populates="products")
```

ForeignKey → правило БД

`brand_id=1` должен указывать на существующий `brands.id`.

relationship → объект Python

`product.brand.name`
→ **North**

↳ relationship не добавляет отдельный столбец brand. Связь в базе хранится через brand_id.

Engine и Session работают вместе

Engine управляет подключениями. Session сопровождает одну работу с данными.

database.py · идея настройки — Учебный пример

```
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(engine)

with SessionLocal() as session:
    product = session.get(Product, 10)
```

Engine · на приложение

Знает драйвер и адрес. Обычно управляет пулом соединений.

Session · на операцию / запрос

Отслеживает ORM-объекты и текущую транзакцию.

↳ Engine можно переиспользовать. Одну Session нельзя одновременно разделять между запросами.

SELECT возвращает объект или None

Ищем по первичному ключу: совпадений будет не больше одного.

Ищем товар: ID 10 · существует ✓ Проверенные результаты

SQLAlchemy 2.0 · фрагмент — Учебный пример

```
stmt = select(Product).where(
    Product.id == 10
)
product = session.scalar(stmt)
```

✓ РЕЗУЛЬТАТ POSTGRESQL

```
Product
id = 10
name = North Runner
price = Decimal("7990.00")
```

↳ Конструктор select описывает запрос. session.scalar выполняет его и берёт первое скалярное значение.

Таблица, схема API и JSON

Три представления одного товара решают разные задачи.

ORM · Product

Столбцы, ключи, связи.

```
price: Decimal
```

Pydantic · ProductRead

Поля публичного ответа.

```
from_attributes=True
```

JSON · ответ

Текст для клиента.

```
"price": "7990.00"
```

Проверенное преобразование — Учебный пример

```
dto = ProductRead.model_validate(product)
payload = dto.model_dump(mode="json")
```

↳ ORM-модель хранит данные, схема API выбирает контракт. Их поля не обязаны совпадать.

add, flush и commit — разные шаги

Посмотрим, когда появился ID и когда строку увидит другое соединение.

Шаг	ID	Виден другому соединению
new	None	Нет
add	None	Нет
flush	32	Нет
commit	32	Да

↳ flush отправляет изменения в текущую транзакцию. commit фиксирует её результат.

У сессии должно быть завершение

Успех фиксируем. Ошибку откатываем. Соединение освобождаем.

Отдельная операция · новая Session — Учебный пример

```
with Session(engine) as session:
    with session.begin():
        session.add(Brand(name="Demo"))
        session.flush()
        # Исключение → rollback
    # Успех → commit
# Выход → close
```

Ошибка после flush

Строка из опыта отсутствует после отката.

После неудачного flush

Перед повторным использованием той же Session вызываем `rollback()`.

↳ Граница транзакции должна иметь одного владельца. close не заменяет commit.

Изменить класс — мало

Код уже ожидает image_url, а в таблице такого столбца ещё нет.

Новая версия модели — Учебный пример

```
image_url: Mapped[str | None] = (
    mapped_column(Text)
)
```

Python ожидает

id · name · price · **image_url**

База пока на версии 0001

id · name · price

image_url отсутствует

✓ ПРОВЕРЕННЫЙ ОТКАЗ

```
SQLSTATE 42703
undefined_column
```

↳ Код и структура базы версионируются согласованно. Новый атрибут не выполняет ALTER TABLE.

Миграция хранит шаг изменения

Alembic применяет явные операции к структуре базы.

0002_image_url.py · исполняемый фрагмент — Учебный пример

```
def upgrade():
    op.add_column("products",
        sa.Column("image_url",
                  sa.Text(),
nullable=True))

def downgrade():
    op.drop_column("products",
"image_url")
```

Было · 0001

У products нет image_url.

Стало · 0002

Столбец появился.
У существующих строк — NULL.

↳ upgrade двигает схему вперёд. downgrade может удалить данные, поэтому его последствия читаем отдельно.

Версии складываются в цепочку

Файлы миграций описывают путь. База помнит применённую ревизию.



В папке учебных примеров — Учебный пример

```
python -m alembic history
python -m alembic current
python -m alembic upgrade head
```

alembic_version

После upgrade: **0002**

head — последняя ревизия нашей линейной цепочки.

↳ Миграция в репозитории и применённая миграция в базе — два разных состояния.

autogenerate предлагает черновик

Alembic сравнивает metadata с базой, а человек проверяет смысл изменений.

После изменения models.py — Учебный пример

```
python -m alembic revision --autogenerate
\
  -m "add image_url"
```

Перед upgrade читаем файл

Имена таблиц · типы · nullable
upgrade · downgrade · сохранность данных

Опасная догадка при rename

```
drop_column("name")
+ add_column("title")
```

Так можно потерять старые названия.

↳ Автогенерация видит различие структур. Намерение переименовать поле она надёжно не угадывает.

Seed добавляет строки после миграции

Сначала появляются таблицы, затем учебные бренды, товары и пользователи.

seed.py · идея вставки бренда — Учебный пример

```
stmt = insert(Brand).values(
    id=1, name="North"
).on_conflict_do_nothing()

with engine.begin() as connection:
    connection.execute(stmt)
```

Повтор seed	Бренды	Товары
Первый запуск	2	3
Второй запуск	2	3

Повторяемость имеет правила

Существующие строки сохраняются.
Seed не сбрасывает корзины и цены.

↳ Миграция управляет структурой. Seed даёт конкретные данные для разработки и проверки.

Слои помогают проследить запрос

Каждый файл отвечает за понятную часть пути от HTTP до данных.

`routers / api.py`	Принять запрос, зависимости, HTTP-ответ
`schemas.py`	Поля входа и публичного ответа
`services.py`	Правила сценария: что разрешено сделать
`models.py`	Таблицы и связи ORM
`database.py + core`	Сессии, подключения и настройки

GET /products/10 → роутер → поиск товара → Product → ProductRead → JSON

↳ Разделение файлов помогает читать и проверять код. Граница транзакции всё равно должна быть явной.

Зачем магазину знать пользователя

Аня и Борис обращаются к одному API, но работают со своими данными.

Аня · user_id=7

Корзина: позиции **55, 56**
Заказ: **501**

Борис · user_id=8

Корзина: позиция **57**
Заказ 501 ему не принадлежит.

Тело запроса не подтверждает личность

```
{"user_id": 7}
```

может написать любой клиент.

↳ Текущего пользователя сервер устанавливает по проверенным учётным данным, а не по заявленному user_id.

Кто ты — и что тебе разрешено

Аутентификация и авторизация отвечают на разные вопросы.

1 · Аутентификация

Токен проверен → пользователь **7**.
«Кто обращается?»

→

2 · Авторизация

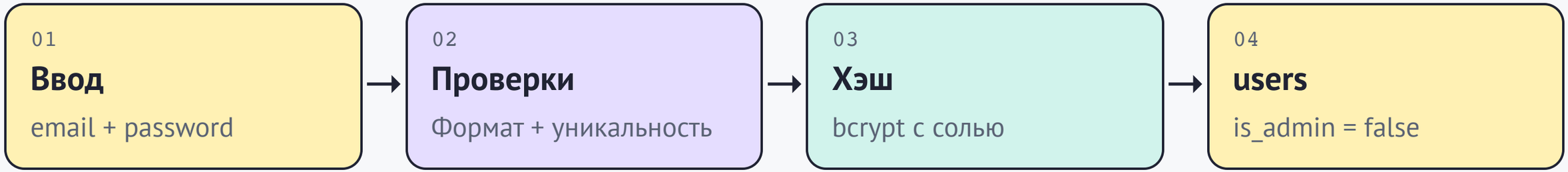
Пользователь 7 владеет заказом 501.
«Разрешено ли это действие?»

Ситуация	Ответ в нашем API
Нет действительных данных входа	401 · требуется аутентификация
Вошёл, но нет роли администратора	403 · действие запрещено
Заказ чужой или отсутствует	404 · не раскрываем чужую запись

↳ Успешный вход подтверждает пользователя. Разрешение проверяется для каждой защищённой операции.

Регистрация создаёт пользователя

Пароль нужен для входа, но не должен попасть в публичный ответ.



POST /auth/register · пример — Учебный пример

```
{"email": "new@example.com",
 "password": "New-demo-N1!"}
```

✓ ФАКТИЧЕСКИЙ ОТВЕТ · 201

```
{
  "id": 11,
  "email": "new@example.com"
}
```

↳ Публичная регистрация не принимает is_admin. Повтор email запрещён также ограничением базы.

Пароль проверяем по хэшу

bcrypt использует случайную соль: один пароль может дать разные хэши.

security.py · фрагмент использования — Учебный пример

```
hashed = pwd_context.hash(password)

pwd_context.verify(password, hashed)
# True

pwd_context.verify("wrong", hashed)
# False
```

✓ ПРОВЕРЕНО С PASSLIB + BCRYPT

```
Хэши одного пароля различаются: True
Верный пароль проходит: True
Неверный пароль проходит: False
```

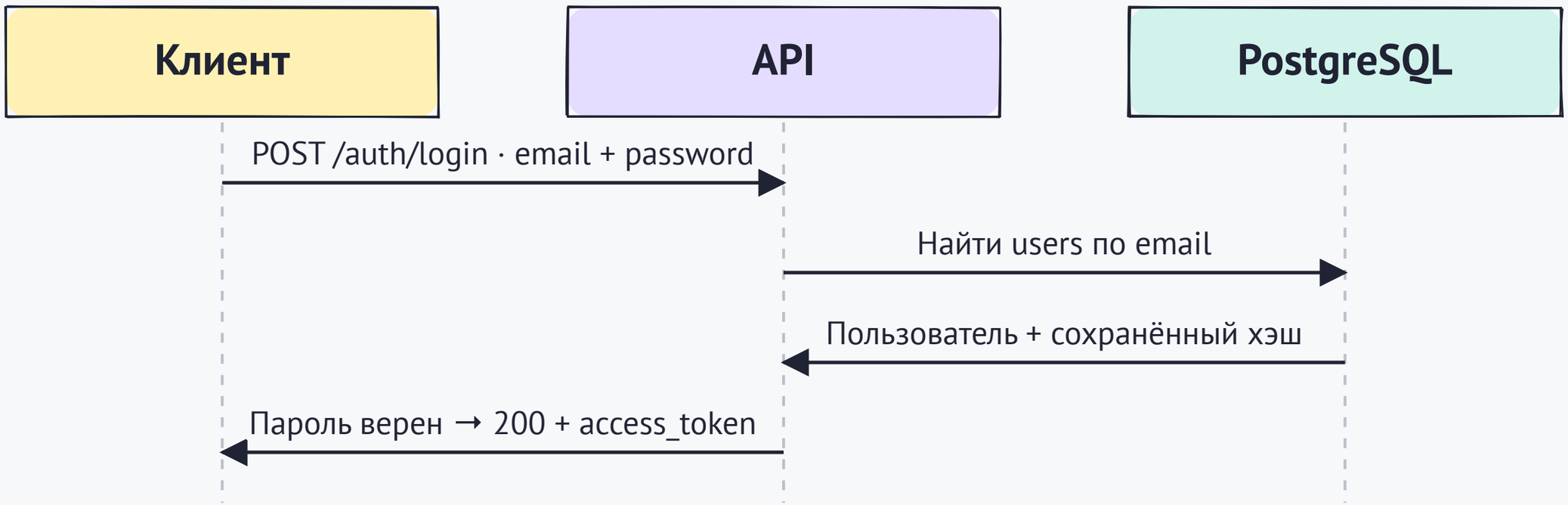
Обратной расшифровки нет

verify повторяет проверку с параметрами из сохранённого хэша.

↳ Храним хэш, а не пароль. Соль не секретна; она сохраняется вместе с параметрами проверки.

Логин выдаёт временный токен

Сначала проверяем пароль, затем создаём подтверждение для следующих запросов.



Неверный пароль или неизвестный email → одинаковый ответ **401**.

↳ Токен выдаём только после проверки. Пароль не кладём в токен и не отправляем в следующих запросах.

JWT: данные и подпись

Подписанный JWT в нашем примере состоит из трёх частей.

`header`
Алгоритм и тип

•

`payload`
Утверждения о доступе

•

`signature`
Проверка целостности

Payload · из проверенного примера — Учебный пример

```
{
  "sub": "7",
  "exp": 1791052678,
  "iss": "sneaker-lesson",
  "aud": "sneaker-api"
}
```

Подпись не скрывает данные

Header и payload можно прочитать.

`exp` — момент истечения в Unix-секундах. Пароля здесь нет.

↳ Проверяем подпись и обязательные поля. Простое декодирование payload не подтверждает подлинность.

Bearer-токен идёт в заголовке

После логина клиент прикладывает токен к защищённому запросу.

Следующий HTTP-запрос · шаблон — Учебный пример

```
GET /auth/me HTTP/1.1
Host: localhost:8007
Authorization: Bearer <access_token>
```

Клиент

Подставляет access_token из ответа логина.

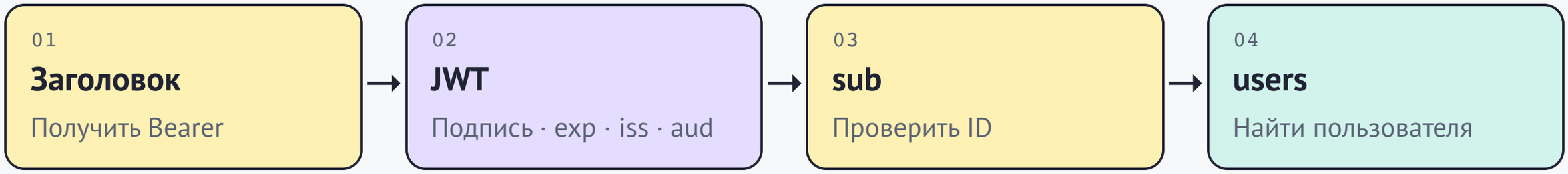
Сервер

Извлекает токен, проверяет его и получает пользователя.

↳ Bearer означает доступ по владению токеном. Его нужно защищать так же внимательно, как данные входа.

get_current_user собирает проверку

Одна зависимость устанавливает пользователя для защищённых маршрутов.



Псевдокод зависимости — Учебный пример

```
claims = проверить_токен(token)
user_id = проверить_ID(claims["sub"])
user = найти_пользователя(user_id)
если user отсутствует: отказ 401
вернуть user
```

Результат зависимости

Объект User из БД.
Следующий шаг проверяет права.

↳ Валидной подписи недостаточно: пользователь должен существовать, а поля токена — пройти проверку.

Администратор — отдельная проверка

Сначала получаем пользователя, затем читаем его роль из базы.

api.py · сокращённый фрагмент — Учебный пример

```
def require_admin(
    user = Depends(get_current_user),
):
    if not user.is_admin:
        raise HTTPException(403)
    return user
```

GET /admin/check	Ответ
Аня · is_admin=false	403
Администратор · is_admin=true	200

Регистрация не выдаёт роль

Публичный запрос с `is_admin=true` отклоняется.

↳ is_admin берём из текущей записи users. Полям запроса и неподтверждённым claims доверять нельзя.

Чужой заказ не становится своим

В запросе должны совпасть и ID заказа, и ID его владельца.

Выборка в GET /orders/{order_id} — Учебный пример

```
stmt = select(Order).where(
    Order.id == order_id,
    Order.user_id == user.id,
)
order = session.scalar(stmt)
if order is None:
    raise HTTPException(404)
```

Аня · user.id = 7

order 501 AND user 7
→ **200 · заказ найден**

Борис · user.id = 8

order 501 AND user 8
→ **404 · строки нет в выборке**

↳ Предсказуемый ID не даёт доступ. Секретность номера заказа не заменяет проверку владельца.

Проверяем не только успешный вход

Меняем одно условие и смотрим, где запрос остановился.

Сценарий	Проверенный ответ
Нет / истёк / неверная подпись	401
Нет пользователя / нет exp / другой алгоритм	401
Действительный токен → /auth/me	200
Обычный пользователь → /admin/check	403
Администратор → /admin/check	200

↳ 401 — не удалось установить пользователя. 403 — пользователь известен, но права не подходят.

Практика: найдите место отказа

Токен → пользователь → права → операция. Где закончится каждый запрос?

Результат	Объяснение
200 · Аня	Токен верен, владелец заказа совпал
404 · Борис	Пользователь найден, заказ ему не принадлежит
401 · истёкший токен	Остановились в get_current_user
403 · обычная роль	Остановились в require_admin

↳ Доступ проверяем до выполнения действия. Следующая глава соединит эти правила с оформлением заказа.